

TP3 - Automatización de calidad y blindaje

Materia: Metodología de Sistemas II

Unidad 3: Verificación y Validación

Alumno: Nicolás Viruel

1. Pre-commit hook (Formatter → Linter → Tests)

Configuración

Script pre-commit-hook.sh (instalación típica: copiar a .git/hooks/pre-commit y chmod +x). Ejecuta en orden estricto sobre el laboratorio de pedidos (TP2):

1. Formatter: `python -m ruff format --check`
2. Linter: `python -m ruff check`
3. Tests: `python -m unittest test_pedidos_descuento.py -v`

Usa `set -e`: ante el primer fallo, el script termina y Git aborta el commit.

Qué muestra la consola si falla un control

Formatter (ejemplo): ver evidencia_precommit_fallo.txt

- Mensaje Would reformat: ... o diff implícito.
- Exit code 1 → Git responde husky/pre-commit o hook estándar: `*commit aborted*`.
- Efecto: el desarrollador debe formatear y volver a intentar; no entra código desalineado al historial local.

Linter: salida tipo F401 ... imported but unused o E722 ... con ruta de archivo; mismo exit 1, commit bloqueado.

Tests: traceback de AssertionError o FAILED (failures=1); el commit no se crea aunque el formato esté bien.

Ejecución exitosa: ver evidencia_precommit_ok.txt - tres pasos en verde y mensaje final Pre-commit OK.

Por qué protege la rama principal

El hook corre antes de que el commit exista; al empujar solo commits ya validados, la CI recibe menos ruido y se reduce el riesgo de merge con tests rotos o estilo inconsistente. Complementa la DoD del TP1 (criterios 1.2 y 1.5) en la máquina del autor.

2. Auditoría estática sobre código de baja calidad

Artefacto: `codigo_baja_calidad.py` (intencionalmente deficiente).

Hallazgos por métrica

Métrica / herramienta — Hallazgo — Severidad

Complejidad ciclomática (radon/ruff) — procesar_pedido: anidamiento > 5, CC > 15 — Alta

Duplicación — Asignación duplicada `cache[str(pedido)] = x` — Media

Duplicación de definición — `enviar_notificacion` definida dos veces — Alta (bug latente)

Dependencias — Uso directo `urllib` sin timeout robusto ni manejo de errores — Media

Mantenibilidad — Parámetros booleanos opacos (`flag`, `otro`) — Alta

Estrategia de refactorización

1. Extraer política de precios a funciones puras (`calcular_total_con_impuestos`, `aplicar_descuento_vip`) → baja CC y habilita tests unitarios como en TP2.
2. Eliminar duplicados y unificar `enviar_notificacion`; un solo punto con inyección de cliente HTTP para tests con mock.
3. Reemplazar flags por tipos expresivos (`enum TipoCliente`, `dataclass Pedido`).
4. Dependencias: fijar versiones en `requirements.lock`, ejecutar `pip-audit` o `dependabot` en CI; sustituir llamadas HTTP crudas por cliente con reintentos acotados.
5. Quality gate: umbral máximo CC = 10 por función en linter (config `ruff/mccabe`) para evitar regresiones.

3. Informe de Calidad en Verde

Documento consolidado de los TPs 1-3. Estado: APTO PARA INTEGRACIÓN (simulación de release v1.4.0-pedidos).

3.1 Trazabilidad PR ↔ Issue

Elemento — Referencia

Issue — #142 - Descuento de envío inconsistente para premium

Pull Request — #87 - Refactor descuento envío + tests AAA

Enlace — Descripción del PR: Closes #142

Labels — bug, ready-for-review

DoD TP1 — Checklist marcada; revisión aprobada por par

3.2 Logs del pre-commit hook

Ejecución local previa al push (extracto): archivo `evidencia_precommit_ok.txt`

- Formatter: 2 archivos conformes
- Linter: All checks passed
- Tests: 6/6 OK

Ningún commit con hash `a1b2c3d` (simulado) se generó mientras el hook fallaba en formato (caso documentado en `evidencia_precommit_fallo.txt`).

3.3 Reporte analítico de cobertura

Módulo — Statements — Branches — Notas

`pedidos_descuento.py` — 100% — 100% — Suite TP2, mapa B1-B10

`test_pedidos_descuento.py` — - — - — 6 casos AAA

Umbral DoD (TP1): $\geq 80\%$ líneas / $\geq 70\%$ ramas → cumplido.

3.4 Diagnóstico final de análisis estático

Área — Resultado post-refactor — Evidencia

Código productivo TP2 — Sin violaciones `ruff` — CI lint verde

Código legacy auditado — Refactor plan aplicado a `procesar_pedido` — Sección 2

Dependencias — Lockfile + auditoría sin CVE críticas — Informe CI `dependency-check`

Certificación

Se certifica que el incremento PR #87 asociado al Issue #142 cumple: trazabilidad, controles pre-commit, cobertura de ramas y plan de deuda técnica sobre el módulo auditado. Recomendación: merge a rama principal tras aprobación final del revisor.

Cierre

La automatización local (hook) acorta el feedback; la auditoría estática prioriza refactor con métricas; el Informe en Verde une trazabilidad, logs, cobertura y lint en una sola decisión de integración.